INF398 - Introducción al aprendizaje automático

Loss functions

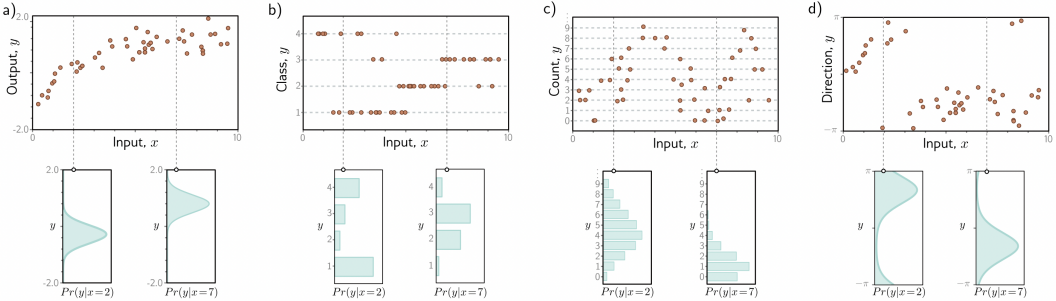
Alejandro Veloz

- Model parameters ϕ determine the mapping from input x to output y.
- **Goal of Training:** Find ϕ that produces the "best possible" mapping.
- **Loss Function $L[\phi]$:** Returns a single number describing the mismatch between predictions $\mathbf{f}[\mathbf{x}_i, \phi]$ and ground-truth $\mathbf{y}_i$.

Maximum Likelihood (MLE)

Another perspective: The model doesn't directly compute y, but a **conditional probability distribution** $P(\mathbf{y} \mid \mathbf{x})$.

1. Choose a parametric distribution $Pr(\mathbf{y} \mid \theta)$.
2. The network computes the parameters $\theta = \mathbf{f}[\mathbf{x}, \phi]$.



Maximum Likelihood criterion

We seek ϕ to maximize the combined probability of all training examples:

$$\Pr\left(\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_I \mid \mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_I\right) = \prod_{i=1}^{I} \Pr\left(\mathbf{y}_i \mid \mathbf{x}_i\right)$$

Maximum Likelihood criterion

We seek ϕ to maximize the combined probability of all training examples:

$$\hat{\boldsymbol{\phi}} = \underset{\phi}{\text{argmax}} \left[\prod_{i=1}^{I} \Pr\left(\mathbf{y}_i \mid \mathbf{x}_i\right) \right]$$

$$= \underset{\phi}{\text{argmax}} \left[\prod_{i=1}^{I} \Pr\left(\mathbf{y}_i \mid \boldsymbol{\theta}_i\right) \right]$$

$$= \underset{\phi}{\text{argmax}} \left[\prod_{i=1}^{I} \Pr\left(\mathbf{y}_i \mid \mathbf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right) \right]$$

Assumptions:

1. Distribution form is the same for all points.
2. Training examples are **independent** (product of probabilities).

Maximizing Log-Likelihood

The product of many small probabilities leads to numerical instability (underflow). **Solution:** Maximize the logarithm of the likelihood.

$$
\begin{aligned}
\hat{\phi} &= \underset{\phi}{\text{argmax}} \left[\prod_{i=1}^{I} \Pr\left(\mathbf{y}_i \mid \mathbf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right) \right] \\
&= \underset{\phi}{\text{argmax}} \left[\log \left[\prod_{i=1}^{I} \Pr\left(\mathbf{y}_i \mid \mathbf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right) \right] \right] \\
&= \underset{\phi}{\text{argmax}} \left[\sum_{i=1}^{I} \log \left[\Pr\left(\mathbf{y}_i \mid \mathbf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right) \right] \right]
\end{aligned}
$$

- Log is monotonically increasing: maximum stays in the same place.
- Converts products into sums.

Negative Log-Likelihood (NLL)

By convention, optimization is framed as **minimization**. We multiply the log-likelihood by -1:

$$L[\phi] = -\sum_{i=1}^{I} \log[Pr(\mathbf{y}_i \mid \mathbf{f}[\mathbf{x}_i, \phi])]$$

This is our final **Loss Function**.

Inference

The network no longer directly predicts the outputs $\mathbf{y}$ but instead determines a probability distribution over $\mathbf{y}$.

We often want a point estimate rather than a distribution, so we return the maximum of the distribution:

$$\hat{\mathbf{y}} = \underset{\mathbf{y}}{\text{argmax}}\left[\Pr\left(\mathbf{y} \mid \mathbf{f}[\mathbf{x}, \hat{\boldsymbol{\phi}}]\right)\right]$$

Recipe for constructing loss functions

1. Choose $\Pr(\mathbf{y} \mid \boldsymbol{\theta})$ defined over the domain of the predictions $\mathbf{y}$ with parameters $\boldsymbol{\theta}$.

2. Set the ML model $\mathbf{f}[\mathbf{x}, \boldsymbol{\phi}]$ to predict one or more of these parameters, so $\boldsymbol{\theta} = \mathbf{f}[\mathbf{x}, \boldsymbol{\phi}]$ and $\Pr(\mathbf{y} \mid \boldsymbol{\theta}) = \Pr(\mathbf{y} \mid \mathbf{f}[\mathbf{x}, \boldsymbol{\phi}])$.

3. (training) Find the network parameters $\hat{\boldsymbol{\phi}}$ that minimize the negative log-likelihood loss function over the training examples $\{\mathbf{x}_i, \mathbf{y}_i\}$:

$$\hat{\boldsymbol{\phi}} = \underset{\boldsymbol{\phi}}{\operatorname{argmin}}[L[\boldsymbol{\phi}]] = \underset{\boldsymbol{\phi}}{\operatorname{argmin}} \left[-\sum_{i=1}^{I} \log \left[\Pr\left(\mathbf{y}_i \mid \mathbf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right)\right] \right]$$

4. (inference) For a new test example $\mathbf{x}$, return either $\Pr(\mathbf{y} \mid \mathbf{f}[\mathbf{x}, \hat{\boldsymbol{\phi}}])$ or the value where this distribution is maximized.

Example 1: Univariate regression

Assume y follows a **Univariate Normal Distribution**:

$$\text{Pr}(y \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y-\mu)^2}{2\sigma^2}\right]$$

If we set the model to predict the mean $\mu = \mathrm{f}[\mathrm{x}, \boldsymbol{\phi}]$:

$$\text{Pr}\left(y \mid \mathrm{f}[\mathrm{x}, \boldsymbol{\phi}], \sigma^2\right) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y-\mathrm{f}[\mathrm{x}, \boldsymbol{\phi}])^2}{2\sigma^2}\right]$$

We aim to find the parameters $\boldsymbol{\phi}$ that make the training data $\{\mathrm{x}_i, y_i\}$ most probable under this distribution.

Deriving Least Squares

To accomplish this, we choose a loss function that minimize the NLL for the Normal distribution:

$$
\begin{aligned}
L[\boldsymbol{\phi}] &= -\sum_{i=1}^{I} \log\left[\Pr\left(y_i \mid \mathsf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right], \sigma^2\right)\right] \\
&= -\sum_{i=1}^{I} \log\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{\left(y_i - \mathsf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right)^2}{2\sigma^2}\right]\right]
\end{aligned}
$$

When we train the model, we seek parameters $\hat{\boldsymbol{\phi}}$ that minimize this loss.

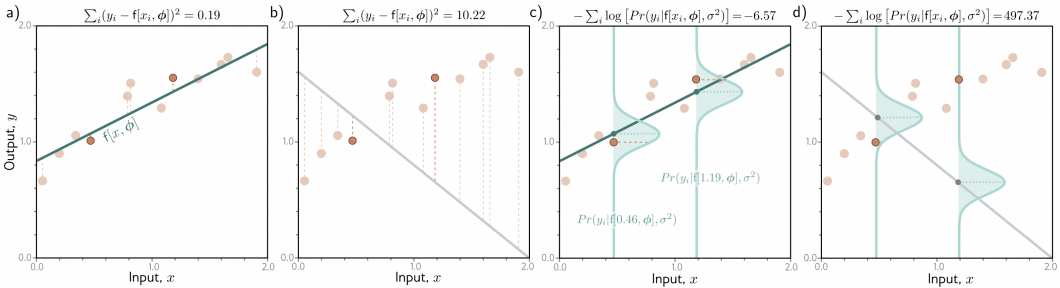
$$
\begin{aligned}
\hat{\phi} &= \underset{\phi}{\mathsf{argmin}} \left[-\sum_{i=1}^{I} \mathsf{log} \left[\frac{1}{\sqrt{2\pi\sigma^2}} \mathsf{exp} \left[-\frac{\left(y_i - \mathsf{f}\left[\mathbf{x}_i, \phi\right]\right)^2}{2\sigma^2} \right] \right] \right] \\
&= \underset{\phi}{\mathsf{argmin}} \left[-\sum_{i=1}^{I} \left(\mathsf{log} \left[\frac{1}{\sqrt{2\pi\sigma^2}} \right] - \frac{\left(y_i - \mathsf{f}\left[\mathbf{x}_i, \phi\right]\right)^2}{2\sigma^2} \right) \right] \\
&= \underset{\phi}{\mathsf{argmin}} \left[-\sum_{i=1}^{I} -\frac{\left(y_i - \mathsf{f}\left[\mathbf{x}_i, \phi\right]\right)^2}{2\sigma^2} \right] \\
&= \underset{\phi}{\mathsf{argmin}} \left[\sum_{i=1}^{I} \left(y_i - \mathsf{f}\left[\mathbf{x}_i, \phi\right]\right)^2 \right]
\end{aligned}
$$

$$L[\phi] = -\sum_{i=1}^{I} \log\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y_i - \mathbf{f}[\mathbf{x}_i, \phi])^2}{2\sigma^2}\right]\right]$$

After removing constants (like σ^2 and 2π):

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \sum_{i=1}^{I} (y_i - \mathbf{f}[\mathbf{x}_i, \phi])^2$$

Least squares assumes independent, normally distributed errors.


a)
$\sum_i (y_i - \mathrm{f}[x_i, \phi])^2 = 0.19$
b)
$\sum_i (y_i - \mathrm{f}[x_i, \phi])^2 = 10.22$
c)
$-\sum_i \log\left[Pr(y_i|\mathrm{f}[x_i, \phi], \sigma^2)\right] = -6.57$
d)
$-\sum_i \log\left[Pr(y_i|\mathrm{f}[x_i, \phi], \sigma^2)\right] = 497.37$
Output, y
Input, x
$\mathrm{f}[x, \phi]$
$Pr(y_i|\mathrm{f}[1.19, \phi], \sigma^2)$
$Pr(y_i|\mathrm{f}[0.46, \phi], \sigma^2)$
0.0
1.0
2.0

Inference

The network no longer directly predicts y but instead predicts the mean $\mu = \mathsf{f}[\mathbf{x}, \boldsymbol{\phi}]$ of the normal distribution over y.

"Best" point estimate $\hat{y}$:

$$\hat{y} = \underset{y}{\text{argmax}} \left[\Pr\left(y \mid \mathsf{f}[\mathbf{x}, \hat{\boldsymbol{\phi}}], \sigma^2 \right) \right]$$

For the univariate normal distribution, the maximum position is determined by the mean parameter μ.

The mean is precisely what the model computed $\rightarrow$ $\hat{y} = \mathsf{f}[\mathbf{x}, \hat{\boldsymbol{\phi}}]$.

Estimating variance

We can treat the variance σ^2 as a **learned parameter** alongside ϕ.

We minimize the full Negative Log-Likelihood (NLL):

$$\hat{\phi}, \hat{\sigma}^2 = \underset{\phi, \sigma^2}{\text{argmin}} \left[-\sum_{i=1}^{I} \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - \mathsf{f}[\mathbf{x}_i, \phi])^2}{2\sigma^2} \right] \right] \right]$$

Inference:

Mean ($\mu = \mathsf{f}[\mathbf{x}, \hat{\phi}]$): The network's primary prediction (the "best guess").

Variance (σ^2): Represents the **model's uncertainty**.

Heteroscedastic Regression

Homoscedastic: Constant variance σ^2 everywhere.

Heteroscedastic: Variance σ^2 changes with input x.

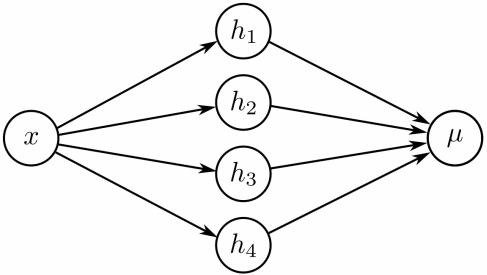
The network predicts two values:

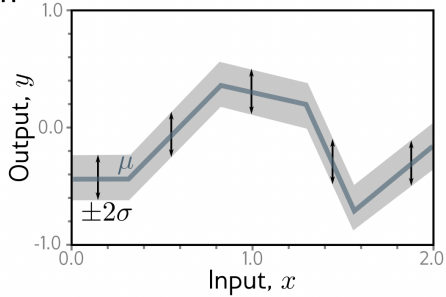
1. Mean: $\mu = \mathbf{f}_1[\mathbf{x}, \phi]$
2. Variance: $\sigma^2 = \mathbf{f}_2[\mathbf{x}, \phi]^2$ (squared to ensure positivity).

This results in the loss function:

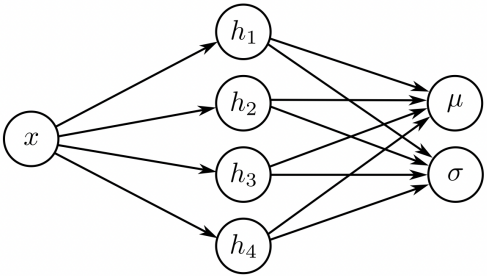
$$\hat{\boldsymbol{\phi}} = \underset{\boldsymbol{\phi}}{\text{argmin}} \left[-\sum_{i=1}^{I} \left(\log \left[\frac{1}{\sqrt{2\pi \mathsf{f}_2 \left[\mathbf{x}_i, \boldsymbol{\phi}\right]^2}} \right] - \frac{\left(y_i - \mathsf{f}_1 \left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right)^2}{2\mathsf{f}_2 \left[\mathbf{x}_i, \boldsymbol{\phi}\right]^2} \right) \right]$$

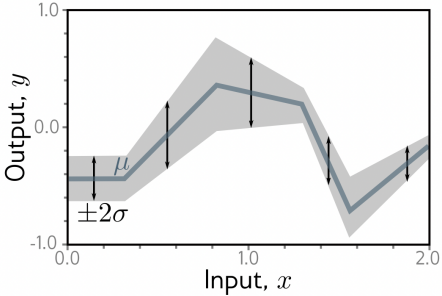
Homoscedastic regression





Heteroscedastic regression





Example 2: Binary classification

Domain $y \in \{0, 1\}$

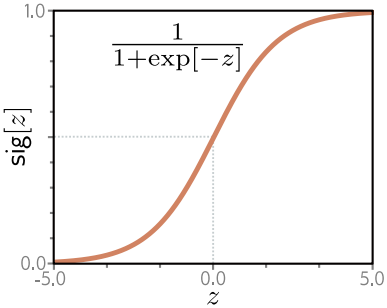
Distribution **Bernoulli**:

$$\Pr(y \mid \lambda) = \begin{cases} 1-\lambda & y = 0 \\ \lambda & y = 1 \end{cases}$$
$$= (1-\lambda)^{1-y}\,\lambda^{y}$$

where $\lambda \in [0, 1]$ (probability of $y = 1$).

To map network output to $[0, 1]$, use the **Logistic Sigmoid**:

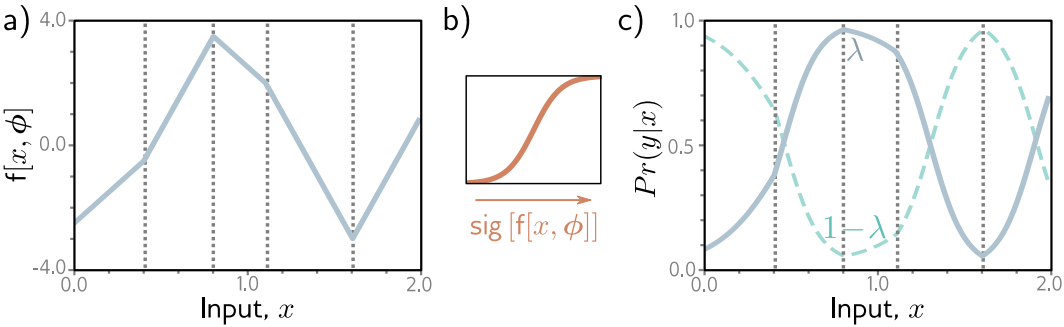
$$\text{sig}[z] = \frac{1}{1 + \exp[-z]}$$



We set the ML model $\text{sig}[\text{f}[\mathbf{x}, \phi]]$ to predict λ.

The likelihood is now:

$$\begin{aligned}\Pr(y \mid \mathbf{x}) &= (1-\lambda)^{1-y} \cdot \lambda^{y} \\ &= (1-\text{sig}[\,\text{f}[\mathbf{x}, \phi]\,])^{1-y} \cdot \text{sig}[\,\text{f}[\mathbf{x}, \phi]\,]^{y}\end{aligned}$$


a)
4.0
0.0
-4.0
f[x, ϕ]
0.0
1.0
2.0
Input, x
b)
sig [f[x, ϕ]]
c)
1.0
0.5
0.0
$Pr(y|x)$
λ
$1-\lambda$
0.0
1.0
2.0
Input, x

Binary cross-entropy loss

The loss function is the negative log-likelihood of the training set:

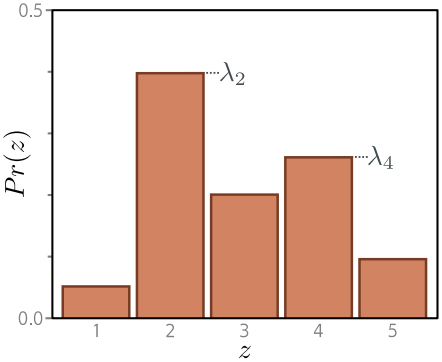
$$L[\boldsymbol{\phi}] = \sum_{i=1}^{I} -(1-y_i)\,\text{log}\left[1-\text{sig}\left[\text{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right]\right] - y_i\,\text{log}\left[\text{sig}\left[\text{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right]\right]$$

Example 3: Multiclass classification

Domain: $y \in \{1, 2, \ldots, K\}$.

Categorical distribution (K parameters $\lambda_1, \lambda_2, \ldots, \lambda_K$).

$$\Pr(y = k) = \lambda_k$$



We set the ML model $\text{softmax}_k[\mathbf{f}[\mathbf{x}, \phi]]$ to predict $\Pr(y = k \mid \mathbf{x}) = \lambda_k$.

Softmax function:

$$\text{softmax}_k[\mathbf{z}] = \frac{\exp[z_k]}{\sum_{k'=1}^{K} \exp[z_{k'}]}$$

Multiclass cross-entropy loss

The loss function is the negative log-likelihood of the training data:

$$L[\phi] = -\sum_{i=1}^{I} \text{log}\left[\text{softmax}_{y_i}\left[\mathbf{f}\left[\mathbf{x}_i, \phi\right]\right]\right]$$

$$= -\sum_{i=1}^{I}\left(\text{f}_{y_i}\left[\mathbf{x}_i, \phi\right] - \text{log}\left[\sum_{k'=1}^{K} \text{exp}\left[\text{f}_{k'}\left[\mathbf{x}_i, \phi\right]\right]\right]\right)$$

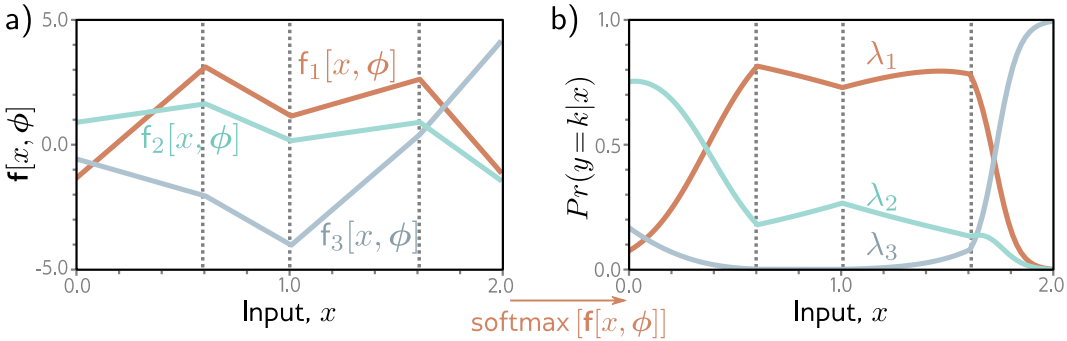
where $\text{f}_{y_i}[\mathbf{x}, \phi]$ and $\text{f}_{k'}[\mathbf{x}, \phi]$ denote the y_i^{th} and k'^{th} outputs of the network, respectively.

Inference

The transformed model output represents a categorical distribution over possible classes $y \in \{1, 2, \ldots, K\}$.

For a point estimate, we take the most probable category $\hat{y} = \text{argmax}_k[\Pr(y = k \mid \mathbf{f}[\mathbf{x}, \hat{\boldsymbol{\phi}}])]$.

This corresponds to whichever curve is highest for that value of $\mathbf{x}$.


a)
5.0
0.0
-5.0
$\mathbf{f}[x, \phi]$
$\mathrm{f}_1[x, \phi]$
$\mathrm{f}_2[x, \phi]$
$\mathrm{f}_3[x, \phi]$
0.0
1.0
2.0
Input, x
softmax $[\mathbf{f}[x, \phi]]$
b)
1.0
0.5
0.0
$Pr(y=k|x)$
λ_1
λ_2
λ_3
0.0
1.0
2.0
Input, x

Multiple outputs

More than one prediction with the same model $\longrightarrow$ the target $\mathbf{y}$ is a vector.

For example:

- Molecule's melting and boiling point (a multivariate regression problem).
- Object class at every point in an image (a multivariate classification problem).

Two options:

1. Define multivariate probability distributions and use a neural network to model their parameters.
2. To treat each prediction as independent.

Multiple outputs

Independence implies that we treat the probability $\Pr(\mathbf{y} \mid \mathbf{f}[\mathbf{x}, \boldsymbol{\phi}])$ as a product of univariate terms for each element $y_d \in \mathbf{y}$:

$$\Pr(\mathbf{y} \mid \mathbf{f}[\mathbf{x}, \boldsymbol{\phi}]) = \prod_d \Pr\left(y_d \mid \mathbf{f}_d[\mathbf{x}, \boldsymbol{\phi}]\right)$$

where $\mathbf{f}_d[\mathbf{x}, \boldsymbol{\phi}]$ is the dth set of network outputs, which describe the parameters of the distribution over y_d.

Negative log probability:

$$L[\boldsymbol{\phi}] = -\sum_{i=1}^{I} \log\left[\Pr\left(\mathbf{y}_i \mid \mathbf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right)\right]$$
$$= -\sum_{i=1}^{I}\sum_{d} \log\left[\Pr\left(y_{id} \mid \mathbf{f}_d\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right)\right]$$

where y_{id} is the dth output from the ith training example.

Two or more prediction types simultaneously

We similarly assume the errors in each are independent.

For example, to predict wind direction and strength:

- Von Mises distribution (defined on circular domains) for the direction and the exponential distribution (defined on positive real numbers) for the strength.
- The joint likelihood of the two predictions is the product of individual likelihoods (by independence assumption).

Distributions for loss functions for different prediction types

Data Type	Domain	Distribution	Use
univariate, continuous, unbounded	$y \in \mathbb{R}$	univariate normal	regression
univariate, continuous, unbounded	$y \in \mathbb{R}$	Laplace or t -distribution	robust regression
univariate, continuous, unbounded	$y \in \mathbb{R}$	mixture of Gaussians	multimodal regression
univariate, continuous, bounded below	$y \in \mathbb{R}^+$	exponential or gamma	predicting magnitude
univariate, continuous, bounded	$y \in [0, 1]$	beta	predicting proportions
multivariate, continuous, unbounded	$\mathbf{y} \in \mathbb{R}^K$	multivariate normal	multivariate regression

Distributions for loss functions for different prediction types

Data Type	Domain	Distribution	Use
univariate, continuous, circular	$y \in (-\pi, \pi]$	von Mises	predicting direction
univariate, discrete, binary	$y \in \{0, 1\}$	Bernoulli	binary classification
univariate, discrete, bounded	$y \in \{1, 2, \dots, K\}$	categorical	multiclass classification
univariate, discrete, bounded below	$y \in \{0, 1, 2, 3, \dots\}$	Poisson	predicting event counts
multivariate, discrete, permutation	$\mathbf{y} \in \text{Perm}[1, 2, \dots, K]$	Plackett-Luce	ranking

Cross-entropy vs. negative log-likelihood

Are they different? **No.**

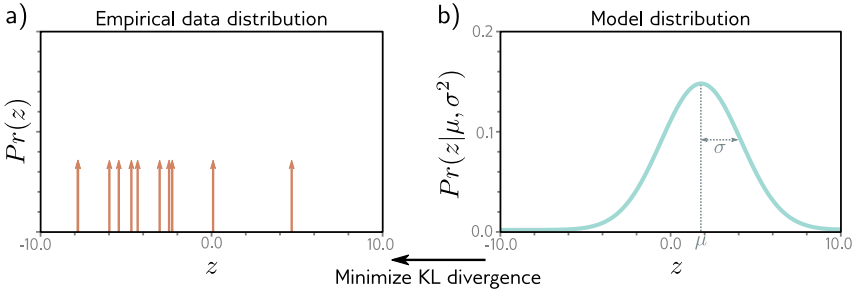
Cross-entropy: Find parameters $\boldsymbol{\theta}$ that minimize the distance (KL divergence) between the empirical data distribution $q(y)$ and a model distribution $\Pr(y \mid \boldsymbol{\theta})$.

MLE: Minimizes the negative log-likelihood of the data.

Mathematically, both lead to the same optimization objective:

$$\hat{\phi} = \underset{\phi}{\text{argmin}} \left[-\sum_{i=1}^{I} \log[Pr(y_i \mid \mathbf{f}[\mathbf{x}_i, \phi])] \right]$$

Cross-entropy loss



a) Empirical distribution of training samples (arrows denote Dirac delta functions).
b) Model distribution (a normal distribution with parameters $\boldsymbol{\theta} = \{\mu, \sigma^2\}$).

In the cross-entropy approach, we minimize the distance (KL divergence) between these two distributions as a function of the model parameters $\boldsymbol{\theta}$.

Kullback-Leibler (KL) divergence

The distance between two probability distributions $q(z)$ and $p(z)$ can be evaluated through:

$$D_{\text{KL}}[q\|p] = \int_{-\infty}^{\infty} q(z)\,\text{log}[q(z)]dz - \int_{-\infty}^{\infty} q(z)\,\text{log}[p(z)]dz$$

Empirical data distribution

The empirical data distribution at points $\{y_i\}_{i=1}^{I}$ can be described as a weighted sum of point masses:

$$q(y) = \frac{1}{I}\sum_{i=1}^{I}\delta\left[y - y_i\right]$$

where $\delta[\bullet]$ is the Dirac delta function.

We want to minimize the KL divergence:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\text{argmin}} \left[\int_{-\infty}^{\infty} q(y) \log[q(y)] dy - \int_{-\infty}^{\infty} q(y) \log[\Pr(y \mid \boldsymbol{\theta})] dy \right]$$

$$= \underset{\boldsymbol{\theta}}{\text{argmin}} \left[- \int_{-\infty}^{\infty} q(y) \log[\Pr(y \mid \boldsymbol{\theta})] dy \right]$$

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\text{argmin}} \left[\underbrace{\cancel{\int_{-\infty}^{\infty} q(y) \log[q(y)] dy}}_{\text{Does not depend on } \theta} - \int_{-\infty}^{\infty} q(y) \log[\Pr(y \mid \boldsymbol{\theta})] dy \right]$$

$$= \underset{\boldsymbol{\theta}}{\text{argmin}} \underbrace{\left[- \int_{-\infty}^{\infty} q(y) \log[\Pr(y \mid \boldsymbol{\theta})] dy \right]}_{\text{Cross-Entropy}}$$

It can be interpreted as the amount of uncertainty that remains in one distribution after taking into account what we already know from the other.

$$
\begin{aligned}
\hat{\boldsymbol{\theta}} &= \underset{\boldsymbol{\theta}}{\text{argmin}} \left[-\int_{-\infty}^{\infty} q(y) \log[\Pr(y \mid \boldsymbol{\theta})] dy \right] \\
&= \underset{\theta}{\text{argmin}} \left[-\int_{-\infty}^{\infty} \left(\frac{1}{I} \sum_{i=1}^{I} \delta\left[y - y_i\right] \right) \log[\Pr(y \mid \boldsymbol{\theta})] dy \right] \\
&= \underset{\boldsymbol{\theta}}{\text{argmin}} \left[-\frac{1}{I} \sum_{i=1}^{I} \log\left[\Pr\left(y_i \mid \boldsymbol{\theta}\right)\right] \right] \\
&= \underset{\boldsymbol{\theta}}{\text{argmin}} \left[-\sum_{i=1}^{I} \log\left[\Pr\left(y_i \mid \boldsymbol{\theta}\right)\right] \right]
\end{aligned}
$$

In machine learning, the distribution parameters $\boldsymbol{\theta}$ are computed by the model $\mathbf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]$, so we have:

$$\hat{\boldsymbol{\phi}} = \underset{\boldsymbol{\phi}}{\text{argmin}} \left[-\sum_{i=1}^{I} \log \left[\Pr\left(y_i \mid \mathbf{f}\left[\mathbf{x}_i, \boldsymbol{\phi}\right]\right)\right] \right]$$

This is precisely the negative log-likelihood criterion.